

1) Explain Alpha–Beta Tree Search and Cutoff Procedure in Detail with Example. [9]

Alpha–Beta Pruning

Alpha–Beta pruning is an optimization technique for the Mini-Max algorithm used in adversarial search and game playing.

It reduces the number of nodes evaluated in the game tree without affecting the final decision.

It is mainly used in two-player games such as:

- Chess
- Tic-Tac-Toe
- Checkers

The algorithm improves efficiency by pruning branches that cannot influence the final decision.

Basic Idea

Mini-Max explores all possible game states.

Alpha–Beta pruning avoids exploring branches that are guaranteed to be worse than previously examined moves.

Two values are maintained:

1. Alpha (α)

- Best value that MAX player can guarantee.
- Initially = $-\infty$

2. Beta (β)

- Best value that MIN player can guarantee.
- Initially = $+\infty$

Pruning occurs when:

```
[
  \beta \leq \alpha
]
\beta \leq \alpha
```

At this point, further exploration is unnecessary.

Working of Alpha–Beta Pruning

MAX Node

- Chooses maximum value
 - Updates alpha value
- ```
[
 \alpha = \max(\alpha, \text{current value})
]
\alpha = \max(\alpha, \text{current value})
```

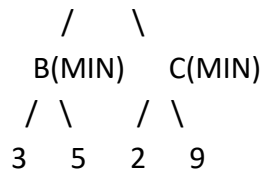
#### MIN Node

- Chooses minimum value
  - Updates beta value
- ```
[
  \beta = \min(\beta, \text{current value})
]
\beta = \min(\beta, \text{current value})
```

Example of Alpha–Beta Pruning

Consider the following game tree:

A(MAX)



Step 1: Evaluate Left Subtree

At node B (MIN):

- Compare 3 and 5
- Minimum = 3

So:

- B returns 3 to A

At A:

- $\alpha = \max(-\infty, 3) = 3$

Step 2: Evaluate Right Subtree

At node C (MIN):

- First child = 2
- $\beta = 2$

Now check condition:

$$[\beta \leq \alpha]$$

$$[2 \leq 3]$$

Condition becomes true.

$$2 \leq 3$$

Hence remaining child (9) is pruned.

Final Decision

Root node A selects:

- $\max(3, 2) = 3$

Therefore best move value = 3.

Cutoff Procedure

Cutoff means stopping search before reaching terminal states.

Instead of exploring entire game tree:

- Search is stopped at fixed depth
- Evaluation function estimates utility

Cutoff Test

Search stops when:

1. Goal state reached
2. Maximum depth reached
3. Time limit exceeded

Evaluation Function

At cutoff depth, heuristic evaluation is applied.

Example in Chess:

- Material count
- Board control
- King safety

Evaluation estimate:

Positive value $\rightarrow$ good for MAX

Negative value $\rightarrow$ good for MIN

Advantages

1. Reduces computation time
2. Searches deeper levels
3. Improves game-playing efficiency
4. Produces same result as Mini-Max

Limitations

1. Performance depends on node ordering
2. Still expensive for large games
3. Requires good heuristic evaluation

Applications

- Chess programs
 - Checkers
 - Decision-making systems
 - Tic-Tac-Toe
 - AI game engines
-

2) What are the issues that need to be addressed for solving CSP efficiently? Explain the solution to them. [9]

Constraint Satisfaction Problem (CSP)

A CSP consists of:

- Variables
- Domains
- Constraints

Goal:

Assign values to variables such that all constraints are satisfied.

Examples:

- Map coloring
- Sudoku
- N-Queen problem

Issues in Solving CSP Efficiently

1. Large Search Space

As variables increase, possible assignments grow exponentially.

Problem

Brute-force search becomes slow.

Solution

Use:

- Backtracking search
- Heuristic methods

2. Variable Selection Problem

Choosing wrong variable increases search time.

Solution: MRV Heuristic - Minimum Remaining Values (MRV):

- Select variable with smallest legal values.

Benefits:

- Detects failure early
- Reduces branching factor

3. Value Ordering Problem

Wrong value choice may lead to conflicts.

Solution: Least Constraining Value (LCV)

Choose value causing minimum restriction on neighboring variables.

Benefits: Keeps future options open

4. Repeated Constraint Checking

Repeated checking wastes computation.

Solution: Constraint Propagation

Reduces domains before assignment.

Techniques:

- Forward Checking
- Arc Consistency

5. Dead-End States

Search may reach inconsistent assignments.

Solution: Backtracking

Undo previous assignments and try alternatives.

Benefits:

- Systematic exploration
- Avoids restarting

6. Constraint Dependency

Variables influence each other strongly.

Solution: Arc Consistency Algorithm

Example: AC-3 algorithm

If value violates constraints:

- Remove it from domain
- This reduces invalid choices early.

7. High Time Complexity

CSPs may become NP-hard.

Solution

Use:

- Heuristics
- Pruning
- Intelligent search

CSP Solving Techniques

1. Backtracking Search

Assign variables one by one.

If conflict occurs:

- Backtrack to previous variable.

2. Forward Checking

After assignment:

- Remove inconsistent values from neighboring domains.
- Detects failure early.

3. Constraint Propagation

Uses constraints repeatedly to reduce domains.

Improves efficiency significantly.

4. Heuristic Methods

MRV

Choose most constrained variable.

Degree Heuristic

Choose variable involved in maximum constraints.

LCV

Choose least constraining value.

Example: Map Coloring

Constraints:

- Adjacent states cannot have same color.

Solutions used:

- MRV
- Forward checking
- Arc consistency

This reduces unnecessary search.

Advantages of Efficient CSP Techniques

1. Reduces search space
2. Faster problem solving
3. Early conflict detection
4. Better scalability

Applications of CSP

- Scheduling
 - Sudoku solving
 - AI planning
 - Timetabling
 - Resource allocation
-

3) Explain in detail the concepts of backtracking and constraint propagation and solve the N-Queen problem using these algorithms. [9]

Backtracking

Backtracking is a systematic search algorithm used to solve CSPs.

It builds solution step-by-step.

If a partial solution violates constraints:

- Algorithm backtracks
- Tries another choice

Steps of Backtracking

1. Start with empty assignment
2. Assign value to variable
3. Check constraints
4. If valid → continue
5. If invalid → backtrack
6. Repeat until solution found

Features

- Depth-first search approach
- Removes invalid partial solutions early
- Memory efficient

Constraint Propagation

Constraint propagation reduces possible values of variables using constraints.

It propagates effects of assignments through the network.

Goal:

Reduce search space before backtracking.

Techniques of Constraint Propagation

1. Forward Checking

After assigning a variable:

- Remove inconsistent values from neighbors.

Example:

If Queen placed in column 1,
remove unsafe positions from other columns.

2. Arc Consistency (AC-3)

For every pair of variables:

- Ensure constraints are satisfied.
Invalid values are removed from domains.

N-Queen Problem

Place N queens on an N×N chessboard such that:

- No two queens attack each other.
Queens attack:
 - Same row
 - Same column
 - Same diagonal

4-Queen Problem Solution

Step 1: Place Queen in Column 1

Q . . .

. . . .

. . . .

. . . .

Unsafe rows and diagonals are removed.

Step 2: Place Queen in Column 2

Try safe position.

Q . . .

. . . Q

. . . .

. . . .

If conflict occurs:

- Backtrack

Step 3: Continue Placement

Final valid arrangement:

. Q . .

. . . Q

Q . . .

. . Q .

Coordinates:

- (1,2)
- (2,4)
- (3,1)
- (4,3)

Working of Backtracking in N-Queen

Algorithm:

1. Place queen in current column
2. Check safety
3. Recursively place next queen
4. If no safe position:
backtrack

Constraint Propagation in N-Queen

After placing each queen:

- Remove attacked rows
- Remove attacked diagonals

This reduces future possibilities.

Example:

If queen at (1,2):

- Row 1 blocked
- Diagonals blocked

Remaining domains shrink.

Advantages

Backtracking

- Simple implementation
- Guaranteed solution

Constraint Propagation

- Reduces search space
- Faster execution
- Early conflict detection

Limitations

Backtracking

- Can become slow for large N

Constraint Propagation

- Extra preprocessing cost

Applications

- | | |
|---------------|------------------|
| • Sudoku | • Puzzle solving |
| • Timetabling | • AI planning |
| • Scheduling | |
-

4) Write a short note on Monte Carlo Tree Search and list its limitations. Write all steps and demonstrate with one example. [9]

Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search is a heuristic search algorithm used for decision-making in games and AI problems.

It combines:

- | | | |
|---------------|---------------------|------------------------|
| • Tree search | • Random simulation | • Statistical sampling |
|---------------|---------------------|------------------------|

MCTS is mainly used in:

- | | | |
|---------|---------------|-----------|
| • Chess | • Tic-Tac-Toe | • Game AI |
| • Go | • Robotics | |

It became popular because of its success in Google DeepMind's AlphaGo.

Basic Idea

Instead of exploring all possible states, MCTS:

1. Randomly simulates games
2. Learns which moves are better
3. Expands promising nodes

The algorithm balances:

- Exploration (trying new moves)
- Exploitation (using known good moves)

Steps of Monte Carlo Tree Search

MCTS consists of four main steps:

1. Selection

Start from root node and select child nodes until a leaf node is reached.

Selection uses UCT formula:

$$[UCT = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}}]$$

Where:

- (w_i) = wins of node

- (n_i) = visits of node
- (N) = parent visits
- (c) = exploration constant

$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

2. Expansion

If selected node is not terminal:

- Add one or more child nodes.

3. Simulation (Rollout)

Play random game from new node until terminal state.

Result may be:

- Win
- Loss
- Draw

4. Backpropagation

Update node statistics:

- Visit count
- Win count

Information propagates back to root.

Example of MCTS

Consider Tic-Tac-Toe.

Initial board:

```
X | O | X
```

```
-----
```

```
O | X |
```

```
-----
```

```
| | O
```

AI must choose next move.

Step 1: Selection

Choose promising move from root.

Suppose move:

- Place X at bottom-left.

Step 2: Expansion

Create new node representing board state.

```
X | O | X
```

```
-----
```

```
O | X |
```

```
-----
```

```
X | | O
```

Step 3: Simulation

Randomly play remaining game.

Suppose AI wins.

Step 4: Backpropagation

Update:

- Visit count +1
- Win score +1

Repeated simulations help choose best move.

Advantages of MCTS

1. Handles large search spaces
2. Does not require complete game tree

Limitations of MCTS

1. High Computational Cost

Needs many simulations for accurate decisions.

2. Slow for Real-Time Systems

Large games may require huge computation time.

3. Random Simulations May Be Weak

Poor rollouts may produce inaccurate results.

4. Memory Consumption

Applications

- | | | |
|-----------------|---------------------|--------------------|
| • AlphaGo | • Robotics planning | • Decision systems |
| • Chess engines | • Path planning | |

3. Produces good decisions with limited time
4. Learns promising moves automatically

Large search trees require more memory.

5. Performance Depends on Number of Simulations

Fewer simulations → poor decision quality.

6. Not Suitable for Deterministic Perfect Evaluation

Games requiring exact calculations may perform poorly.

5) Apply Constraint Satisfaction Method to Solve: SEND + MORE = MONEY

Constraint Satisfaction Problem (CSP)

In CSP:

- Variables represent letters
- Digits are domains
- Constraints ensure valid arithmetic solution

Problem

SEND
+ MORE

MONEY

Each letter represents a unique digit.

Variables

[S,E,N,D,M,O,R,Y]

Domains:

[{0,1,2,3,4,5,6,7,8,9}]

Constraint:

- No two letters have same digit.
- Leading digits cannot be zero.

So:

- $(S \neq 0)$
- $(M \neq 0)$

Step-by-Step CSP Solution

From arithmetic:

SEND

MORE

MONEY

Since result has 5 digits:

- (M = 1)

Carry analysis gives:

- (O = 0)

Using constraints and column-wise addition:

Final assignment:

Letter	Digit
S	9
E	5
N	6
D	7
M	1
O	0
R	8
Y	2

Verification

9567

+ 1085

10652

Thus:

SEND = 9567

MORE = 1085

MONEY = 10652

Hence CSP solution satisfies all constraints.

CSP Techniques Used

1. Backtracking
2. Constraint propagation
3. Domain reduction
4. Consistency checking

Applications

- Cryptarithmic puzzles
- Scheduling
- Timetabling
- Sudoku

6) List all problem-solving strategies. What is backtracking? Explain with N-Queen problem using backtracking. [8]

Problem-Solving Strategies in AI

AI uses several strategies to solve problems systematically.

Types of Problem-Solving Strategies

1. Generate and Test

Generate possible solutions and test them.

2. Hill Climbing

Move toward better state using heuristic.

3. Best First Search

Expand most promising node first.

4. A* Search

Uses:

- Path cost
- Heuristic estimate

Backtracking

Backtracking is a recursive search algorithm used to solve CSPs.

It builds solution step-by-step.

If a partial solution violates constraints:

- Undo last step
- Try another possibility

It follows Depth First Search (DFS).

Steps of Backtracking

1. Start with empty solution
2. Choose variable
3. Assign value
4. Check constraints
5. If valid → continue
6. Else → backtrack
7. Repeat until solution found

N-Queen Problem

Place N queens on chessboard such that:

- No two queens attack each other.

Queens must not share:

- Row
- Column
- Diagonal

4-Queen Problem

Goal:

Place 4 queens safely on 4×4 board.

Step-by-Step Backtracking

Step 1: Place Queen in Column 1

Q . . .

. . . .

. . . .

. . . .

Step 2: Place Queen in Column 2

Try safe position.

Q . . .

. . . .

. Q . .

....

If conflict occurs later:

- Backtrack

Step 3: Continue Search

Final valid solution:

. Q . .

. . . Q

Q . . .

. . Q .

Coordinates:

- (1,2)
- (2,4)
- (3,1)
- (4,3)

Backtracking Algorithm for N-Queen

1. Place queen in current column
2. Check safety:
 - Row safe?
 - Upper diagonal safe?
 - Lower diagonal safe?
3. If safe:
place queen
4. Recur for next column
5. If no position works:
backtrack

Advantages of Backtracking

1. Simple and systematic
2. Reduces unnecessary search
3. Guarantees solution if exists
4. Widely used in CSPs

Limitations

1. Time-consuming for large problems
2. May explore many states
3. Exponential complexity

Applications

- Sudoku
- Scheduling
- CSP problems
- Maze solving
- Puzzle solving

7) (i) Explain limitations of game search algorithm, define game theory, differentiate between stochastic and partially observable games with examples. [9]

Game Theory

Game theory is a mathematical approach used to study strategic interactions between intelligent players where the outcome depends on actions of all participants.

In AI, game theory helps in:

- Decision making
- Competitive problem solving
- Chess
- Adversarial search
- Examples:
- Poker
- Tic-Tac-Toe

Limitations of Game Search Algorithms

Game search algorithms such as Mini-Max and Alpha–Beta pruning face several limitations.

1. Large Search Space

Games like Chess have enormous possible moves.

Example:

- Chess has billions of game states.

Problem:

- Complete search becomes impractical.

2. High Time Complexity

Searching deep game trees requires huge computation time.

Mini-Max complexity:

$[O(b^d)]$

Where:

- (b) = branching factor
- (d) = depth

$O(b^d)$

3. Memory Consumption

Large trees require large memory for storing states.

4. Imperfect Evaluation Function

Heuristic evaluation may not always predict best move correctly.

5. Real-Time Constraints

Some games require fast decisions.

Algorithms may not finish search in time.

6. Horizon Effect

Algorithm may fail to detect future danger beyond search depth.

7. Uncertainty Handling

Traditional search algorithms work poorly with:

- Random events
- Hidden information

Stochastic Games

Stochastic games involve randomness or probability.

Chance factors affect outcomes.

Examples:

- Dice games
- Poker
- Backgammon

Features:

- Uses probability
- Includes chance nodes
- Outcomes uncertain

Partially Observable Games

In partially observable games, players cannot see complete game state.

Some information remains hidden.

Examples:

- Poker
- Battleship

Features:

- Incomplete information
- Requires belief/state estimation
- Hidden opponent actions

Difference Between Stochastic and Partially Observable Games

Feature	Stochastic Games	Partially Observable Games
Definition	Games with randomness	Games with hidden information
Uncertainty Source	Chance events	Incomplete visibility

Feature	Stochastic Games	Partially Observable Games
Uses Probability	Yes	Sometimes
Information Available	Mostly visible	Partially hidden
Example	Backgammon	Poker
Chance Nodes	Present	May or may not exist

(ii) Explain how use of alpha and beta cutoffs improve performance of Mini-Max algorithm. [9]

Mini-Max Algorithm

Mini-Max is used in two-player adversarial games.

- MAX player tries to maximize score.
- MIN player tries to minimize score.

Problem:

Mini-Max explores entire game tree, which is computationally expensive.

Alpha–Beta Pruning

Alpha–Beta pruning improves Mini-Max by removing unnecessary branches.

It gives same final result as Mini-Max but with less computation.

Alpha and Beta Values

Alpha (α)

Best value MAX player can guarantee.

Initially:

$\alpha = -\infty$

$\alpha = -\infty$

Beta (β)

Best value MIN player can guarantee.

Initially:

$\beta = +\infty$

$\beta = +\infty$

Cutoff Condition

Pruning occurs when:

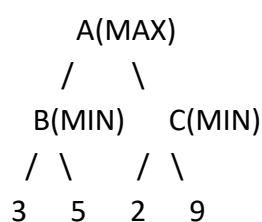
$\beta \leq \alpha$

$\beta \leq \alpha$

At this point:

- Remaining branches cannot affect final decision.
- Search stops for that branch.

Example



Step 1

At B(MIN):

- $\min(3,5)=3$

A receives:

- $\alpha = 3$

Step 2

At C(MIN):

- First child = 2
- $\beta = 2$

Now:

$2 \leq 3$

Condition true.

$2 \leq 3$

Therefore node 9 is pruned.

Performance Improvement

1. Reduces Nodes Evaluated

Unnecessary branches are ignored.

2. Faster Decision Making

Less computation time.

Time Complexity

Without pruning:

$[O(b^d)]$

With Alpha–Beta pruning:

$[O(b^{d/2})]$

(best case)

$O(b^{d/2})$

Applications

- Chess engines
- Tic-Tac-Toe
- Checkers
- AI game playing

3. Allows Deeper Search

More levels can be explored in same time.

4. Same Optimal Result

Produces same output as Mini-Max.

8) What is Constraint Satisfaction Problem? State the types of consistencies. Solve: $BASE + BALL = GAMES$ [6]

Constraint Satisfaction Problem (CSP)

A CSP consists of:

1. Variables
2. Domains
3. Constraints

Goal:

Assign values to variables such that all constraints are satisfied.

Examples:

- Sudoku
- Map coloring
- Cryptarithmic

Types of Consistencies

1. Node Consistency

Each variable satisfies unary constraints.

2. Arc Consistency

For every value of one variable, compatible value exists in connected variable.

Example:

AC-3 algorithm.

3. Path Consistency

Consistency maintained among three variables.

4. k-Consistency

Any consistent assignment of $k-1$ variables can be extended to k th variable.

Crypt Arithmetic Problem

BASE

+ BALL

GAMES

Each letter represents unique digit.

Variables

[

B,A,S,E,L,G,M

]

Constraints:

- Digits unique
- Leading digits non-zero

CSP Solution

Using:

- Backtracking
- Constraint propagation
- Column-wise addition

Possible valid assignment:

Letter	Digit
B	7
A	4
S	8
E	3
L	5
G	1
M	9

Verification

BASE = 7483

BALL = 7455

GAMES = 14938

Check:

[7483 + 7455 = 14938]

7483 + 7455 = 14938

Hence constraints are satisfied.

9) Write Mini-Max search algorithm for two players. How use of Alpha and Beta cutoffs improve performance? [9]

Mini-Max Algorithm

Mini-Max is an adversarial search algorithm used in two-player games.

Players:

- MAX → maximizes utility
 - MIN → minimizes utility
- Examples:
- Chess
 - Tic-Tac-Toe

Working Principle

1. Generate game tree
2. Evaluate terminal states
3. Propagate values upward
4. MAX chooses maximum
5. MIN chooses minimum

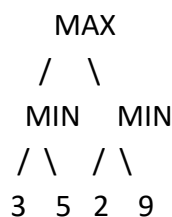
Mini-Max Algorithm

Algorithm Steps

MINIMAX(state)

1. If terminal state:
 return utility value
2. If MAX player's turn:
 best = $-\infty$
 for each child:
 best = max(best, minimax(child))
 return best
3. If MIN player's turn:
 best = $+\infty$
 for each child:
 best = min(best, minimax(child))
 return best

Example



MIN nodes:

- Left MIN = 3
- Right MIN = 2

MAX chooses:

$\max(3, 2) = 3$

$\max(3, 2) = 3$

Final answer = 3.

Alpha-Beta Cutoffs

Alpha-Beta pruning improves Mini-Max efficiency.

Alpha Value

Best value MAX can achieve.

$\alpha = -\infty$

Beta Value

Best value MIN can achieve.

$$[\beta = +\infty]$$

Cutoff Condition

Pruning occurs when:

$$[\beta \leq \alpha]$$

$$\beta \leq \alpha$$

Performance Improvement

1. Fewer Nodes Evaluated

Unnecessary branches skipped.

2. Faster Search

Reduced execution time.

Complexity

Mini-Max:

$$[O(b^d)]$$

Alpha-Beta:

$$[O(b^{d/2})]$$

(best case)

$$O(b^d)$$

$$O(b^{d/2})$$

Applications

- Chess AI
 - Checkers
 - Game playing systems
 - Decision making
-

3. Deeper Search Possible

More levels explored within same time.

4. Same Result as Mini-Max

Optimal move remains unchanged.

10) Explain Mini-Max and Alpha-Beta Pruning Algorithm for Adversarial Search with Example. [9]

Adversarial Search

Adversarial search is used in environments where:

- Multiple players compete
- One player's gain is another player's loss

Examples:

- Chess
- Tic-Tac-Toe
- Checkers

AI uses adversarial search to choose the best move against an opponent.

Mini-Max Algorithm

Mini-Max is a decision-making algorithm used in two-player games.

Players:

- MAX → tries to maximize score
- MIN → tries to minimize score

The algorithm assumes:

- Both players play optimally.

Working of Mini-Max

1. Generate game tree
2. Evaluate terminal states
3. Propagate utility values upward
4. MAX selects maximum value
5. MIN selects minimum value

Mini-Max Formula

At MAX node:

$[V(s) = \max(V(\text{child}))]$

$V(s) = \max(V(\text{child}))$

At MIN node:

$[V(s) = \min(V(\text{child}))]$

$V(s) = \min(V(\text{child}))$

Example of Mini-Max



Step 1: Evaluate MIN Nodes

At B:

- $\min(3, 5) = 3$

At C:

- $\min(2, 9) = 2$

Step 2: Evaluate MAX Node

At A:

- $\max(3, 2) = 3$
 $\backslash \max(3, 2) = 3$

Final decision = 3.

Limitations of Mini-Max

1. Explores entire tree
2. High computation time
3. Large memory requirement

Complexity:

$[O(b^d)]$

Where:

- (b)=branching factor
- (d)=depth

$O(b^d)$

Alpha-Beta Pruning

Alpha-Beta pruning improves Mini-Max performance by pruning unnecessary branches.

It gives:

- Same result as Mini-Max
- Less computation

Alpha and Beta Values

Alpha (α)

Best value MAX can guarantee.

$[\alpha = -\infty]$

$\backslash \alpha = -\infty$

Beta (β)

Best value MIN can guarantee.

$[\beta = +\infty]$

$\backslash \beta = +\infty$

Cutoff Condition

Pruning occurs when:

$$[\beta \leq \alpha]$$

$$\beta \leq \alpha$$

Example of Alpha–Beta Pruning



At B:

- $\min(3,5)=3$
- $\alpha=3$

At C:

- first child=2
- $\beta=2$

Now:

$$[2 \leq 3]$$

$$2 \leq 3$$

So node 9 is pruned.

Advantages of Alpha–Beta Pruning

1. Reduces nodes evaluated
2. Faster execution
3. Enables deeper search
4. Same optimal result

Best-case complexity:

$$[O(b^{d/2})]$$

$$O(b^{d/2})$$

Applications

- Chess AI
- Game playing systems
- Robotics
- Strategic decision making

11) Define and Explain Constraint Satisfaction Problem (CSP). [9]

Constraint Satisfaction Problem (CSP)

A CSP is a problem where values are assigned to variables while satisfying given constraints.

A CSP consists of:

1. Variables
2. Domains
3. Constraints

Goal:

Find assignment satisfying all constraints.

Components of CSP

1. Variables

Objects requiring values.

Example:

- X, Y, Z

2. Domains

Possible values for variables.

Example:

$[D(X)=\{1,2,3\}]$

$D(X)=\{1,2,3\}$

3. Constraints

Rules restricting assignments.

Example:

$[X \neq Y]$

$X \neq Y$

Example of CSP

Map Coloring Problem

Goal: Color neighboring states differently.

Variables:

- States

Domains:

- {Red, Green, Blue}

Constraint:

Adjacent states cannot have same color.

Types of Constraints

1. Unary Constraint

Single variable involved.

Example:

- $X \neq 1$

2. Binary Constraint

Two variables involved.

Example:

- $X \neq Y$

3. Global Constraint

Multiple variables involved.

Example:

- All variables different

CSP Solving Techniques

1. Backtracking

Assign values step-by-step.

If conflict occurs:

- Undo assignment
- Try another value

2. Forward Checking

Removes inconsistent values from neighboring domains.

3. Constraint Propagation

Reduces domains using constraints.

Example:

- Arc consistency (AC-3)

4. Heuristics

MRV (Minimum Remaining Values)

Choose variable with fewest legal values.

Degree Heuristic

Choose variable with most constraints.

Least Constraining Value

Choose value affecting others least.

Types of Consistency**Node Consistency**

Single variable consistency.

Arc Consistency

Consistency between variable pairs.

Path Consistency

Consistency among three variables.

Applications of CSP

- Sudoku
 - Timetabling
 - Scheduling
 - Cryptarithmic
 - N-Queen problem
-

Advantages

1. Efficient problem solving
 2. Reduces search space
 3. Suitable for combinatorial problems
-

Limitations

1. NP-hard for large problems
 2. Large search space
 3. Complex constraints increase difficulty
-

12) Explain with Example Graph Coloring Problem. [9]**Graph Coloring Problem**

Graph coloring is a CSP where colors are assigned to graph vertices such that adjacent vertices have different colors.

Goal:

Minimize number of colors while satisfying constraints.

Components**Vertices**

Represent objects/states.

Edges

Represent relationships between vertices.

Colors

Possible assignments.

Constraint

Adjacent vertices cannot have same color.

If:

- Vertex A connected to B

Then:

[

$\text{Color}(A) \neq \text{Color}(B)$

]

$\text{Color}(A) \neq \text{Color}(B)$

Example

Consider graph:

A
/\n
B---C

Available colors:

- Red
 - Green
 - Blue
-

Coloring Solution

Vertex	Color
A	Red
B	Green
C	Blue

Constraints satisfied because adjacent nodes have different colors.

Graph Coloring as CSP

Variables

Vertices:

- A, B, C

Domains

- {Red, Green, Blue}

Constraints

- $A \neq B$
 - $B \neq C$
 - $A \neq C$
-

Algorithm Used

Backtracking

1. Assign color
 2. Check constraints
 3. If conflict:
backtrack
-

Applications

1. Map coloring
 2. Timetabling
 3. Register allocation in compilers
 4. Frequency assignment
 5. Scheduling
-

Advantages

1. Efficient resource allocation
 2. Reduces conflicts
 3. Useful in optimization
-

Limitations

1. Complex for large graphs
 2. NP-complete problem
 3. Requires intelligent heuristics
-

13) How AI Technique is used to solve Tic-Tac-Toe Problem. [9]

Tic-Tac-Toe Problem

Tic-Tac-Toe is a two-player game played on a 3×3 board.

Players:

- X
- O

Goal:

Place three marks in:

- Row
- Column
- Diagonal

AI Techniques Used

AI solves Tic-Tac-Toe using:

1. Game tree
2. Adversarial search
3. Mini-Max algorithm
4. Heuristic evaluation

Game Tree Representation

Each board configuration represents a state.

Possible moves generate child nodes.

Root node:

- Current board

Leaf nodes:

- Win
- Lose
- Draw

Mini-Max in Tic-Tac-Toe

MAX Player

AI tries to maximize score.

MIN Player

Opponent tries to minimize score.

Utility values:

- Win = +1
- Draw = 0
- Loss = -1

Working Steps

1. Generate possible moves
2. Build game tree
3. Apply Mini-Max
4. Choose best move

Example

Current board:

X | O | X

O | X |

| | O

AI evaluates moves.

Suppose AI places X at bottom-left:

X | O | X

O | X |

X | | O

AI wins diagonally.

Use of Alpha-Beta Pruning

Alpha-Beta pruning reduces unnecessary exploration.

Benefits:

- Faster decision making
- Reduced computation

Heuristic Evaluation

If full search impossible:

- Estimate board quality

Example factors:

- Winning chances
- Blocking opponent
- Two-in-a-row positions

Advantages of AI in Tic-Tac-Toe

1. Optimal decisions
2. Never loses with correct play

Applications

- Game AI
- Robotics planning

3. Fast computation
4. Demonstrates adversarial search

- Decision systems
- Educational AI models